

Session 3.1

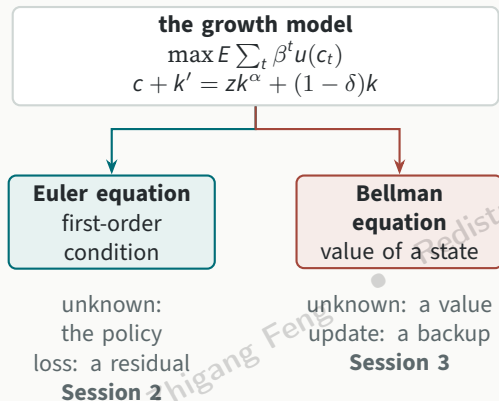
The Growth Model, With One Thing Hidden

Dynamic programming works, and says exactly what it needs; this deck is what happens when it cannot have it



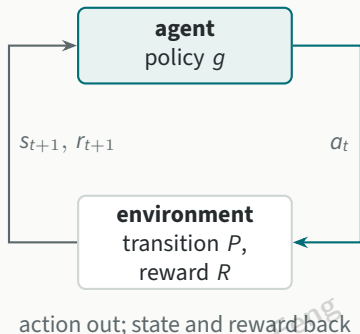
Zhigang Feng

Session 2 Took the Euler Road. The Bellman Equation Is the Other One



- ▶ Session 2 trained a policy network on Euler residuals. It never approximated a value.
- ▶ The Bellman equation describes the same optimum through the *value* of each state.
- ▶ It needs no derivatives: discrete choices and binding constraints are fine.
- ▶ **This session takes that road, and asks what survives when the model cannot be written down.**

Learning by Acting: the Third Way to Learn



- ▶ **Reinforcement learning**: act, observe a reward and a new state, adjust the rule.
- ▶ Session 2 had a label at every input. Here there is only a *reward*: one number, often late.
- ▶ **Credit assignment**: which of many past choices earned it?
- ▶ **Every algorithm in this session answers that question.**

Source: R. Sutton, A. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., 2018, ch. 1 and 3.

Session 3.1

The Benchmark, and What It Requires

Dynamic programming works, and says exactly what it needs



The Growth Model: a Sequence Problem, and Its Recursive Form

sequence problem

$$\max E_0 \sum_t \beta^t \ln c_t$$

$$c_t + k_{t+1} = y(k_t, z_t)$$

one function, not a path

Bellman equation

$$V = \max_{k'} \{ \ln c + \beta E[V' \mid z] \}$$

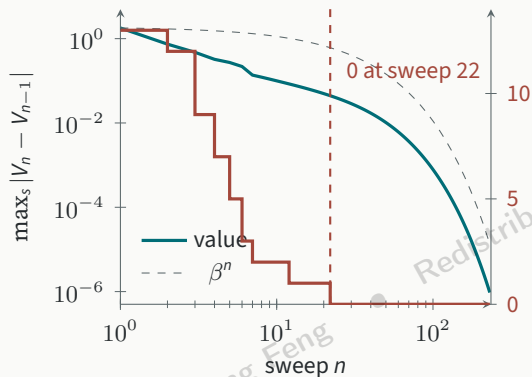
$$y(k, z) = zk^\alpha + (1 - \delta)k$$

output plus capital left

- ▶ k is a *real number*. So is the choice: next capital k' , anywhere in $(0, y)$.
- ▶ Equivalently a **saving share** $a \in (0, 1)$, with $k' = ay$ and $c = (1 - a)y$ — positive consumption at every a , which is Session 2's device.
- ▶ Two shocks only, $z \in \{0.9, 1.1\}$, each staying with probability 0.8.
- ▶ **Nothing here mentions a grid. The grid arrives with the algorithm, in two frames' time.**

Source: Miao & Feng, *AI for Economists*, ch. 11, §11.1. Calibration throughout Session 3 is the book's — $\alpha = 0.36$, $\beta = 0.95$, $\delta = 0.10$, $u = \ln c$, $z \in \{0.9, 1.1\}$ with switching probability 0.2 — not Sessions 1–2's $\alpha = 0.33$, $\beta = 0.96$.

The Benchmark: Dynamic Programming, as Session 1 Left It



- ▶ Session 1.4: T is a contraction of modulus β , so **value function iteration** finds its one fixed point.
- ▶ Red, right axis: states where the greedy rule $g_n = \arg \max_{k'} W(k')$ is still wrong — 13, 12, 9, 7, 5, 3, then two for five sweeps and *one* for ten more, zero at 22. The value needs 231 sweeps.
- ▶ **The residual is not the policy. Deck 3.6 shows the converse, and it is worse.**

Source: Session 3 lab, block A; 7×2 grid, $\alpha = 0.36$, $\beta = 0.95$, $\delta = 0.10$, $u = \ln c$, $z \in \{0.9, 1.1\}$ with switching probability 0.2.

Value Function Iteration, Line by Line

Value function iteration on a grid $\mathcal{K} \times \mathcal{Z}$

```
1 initialise  $V_0(k, z) \leftarrow 0$ 
2 repeat
3   for every grid point  $(k, z)$ :
4     for every  $k' \in \mathcal{K}$  with  $c = zk^\alpha + (1 - \delta)k - k' > 0$ :
5       score it:  $W(k') \leftarrow u(c)$ 
6       add tomorrow:  $W(k') \leftarrow W(k') + \beta \sum_{z'} P_z(z' | z) V_n(k', z')$ 
7      $V_{n+1}(k, z) \leftarrow \max_{k'} W(k')$ ,  $g_{n+1}(k, z) \leftarrow \arg \max_{k'} W(k')$ 
8 until  $\max_{(k, z)} |V_{n+1} - V_n| < \text{tol}$ 
```

- ▶ **Line 4** decides which actions exist (f, δ); **line 5** scores one (u); **line 6** averages over tomorrow (P_z). Three reads, three jobs — and **sampling removes only line 6's sum**.
- ▶ **Line 3 is the grid: DP discretises the state because it must sweep it.**

Source: Miao & Feng, *AI for Economists*, ch. 11, “Value function iteration: how DP computes”.

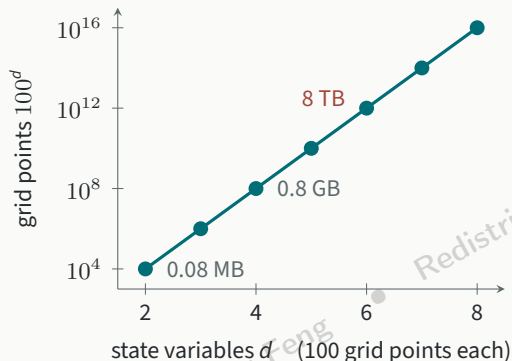
What Dynamic Programming Quietly Requires

Requirement	In the algorithm	Where it fails
to enumerate the model, to score the model, to average	line 4: which k' keep $c > 0$ line 5: $u(c)$ line 6: the sum over z' uses P_z	actions you are only offered rewards you can only observe prices that depend on a distribution
a state space to sweep	line 3: every (k, z)	m^d points with d state variables
nothing to try out	the max inspects every action	samples show only what you tried

- **In two dimensions none of this binds. The requirement that breaks first is not the grid.**

Source: Miao & Feng, *AI for Economists*, ch. 11, “Value function iteration: how DP computes”.

Challenge 1: the Curse of Dimensionality in the State



- ▶ A table with m points per dimension has m^d entries.
- ▶ Go: rules on one page, about 10^{170} positions. A known model, and no table.
- ▶ *Sampling changes which states get visited, not how many numbers a table needs.* Decks 3.5 and 3.6 answer this one.
- ▶ **And the curse has a second source, which appears the moment we improve a learner — Deck 3.2.**

Source: Memory at 8 bytes per entry. Miao & Feng, *AI for Economists*, ch. 11; K. Judd, *Numerical Methods in Economics*, 1998.

Session 3.1

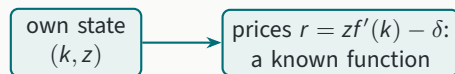
What We Pretend Not to Know

The environment you cannot write down, and the weakest form of not knowing

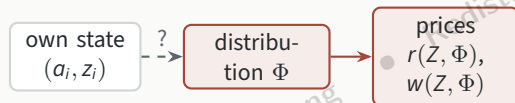


Challenge 2: an Environment You Cannot Write Down

representative agent



heterogeneous agents

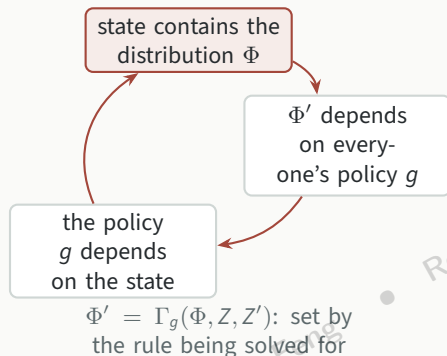


$\Phi' = \Gamma_g(\Phi, Z, Z')$: set by everyone's saving rule

- ▶ Growth model: the household's own state pins down prices.
- ▶ Heterogeneous agents with aggregate shocks: prices depend on the wealth distribution Φ .
- ▶ Its law of motion Γ_g has no closed form in general. It depends on the policy being solved for.
- ▶ **Prices are the environment. Here they must be learned, not specified.**

Source: P. Krusell, A. Smith, *JPE* 106(5), 1998; W. Den Haan, *JEDC* 34(1), 2010; Miao & Feng, ch. 11.

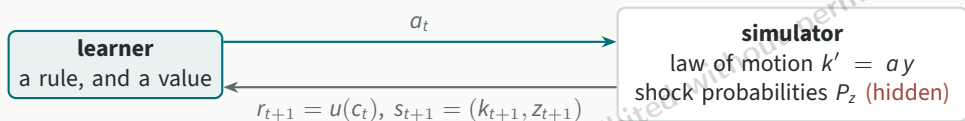
Where the Formula Runs Out — and Two Ways to Write It Anyway



- ▶ Writing Φ into the state makes the problem recursive, and infinite-dimensional.
- ▶ **The law of motion can still be written:** in continuous time by the Kolmogorov forward equation, in discrete time by an analytic, *differentiable* kernel — Deck 3.6.
- ▶ **But an equation for Γ_g is not a table for it:** the state is still a distribution, so challenge 1 remains. Krusell and Smith keep a few moments.
- ▶ **Both challenges at once: a law of motion you must construct, on a state you cannot sweep.**

Source: Aiyagari, *QJE* 109(3), 1994; Krusell & Smith, *JPE* 106(5), 1998; Achdou et al., *REStud* 89(1), 2022. The two-point split is Young, *JEDC* 34(1), 2010; using it as an analytic, differentiable law of motion is Feng, Han, Sargent & Zhu (2026, working paper).

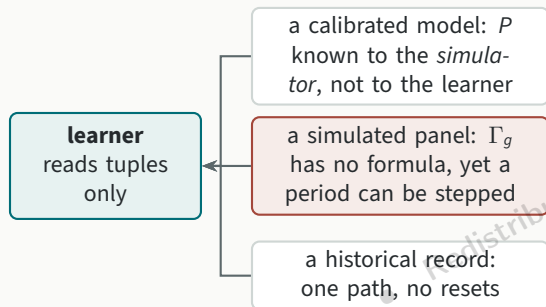
What We Pretend Not to Know: the Shock Probabilities



- ▶ **Hidden from the learner:** the probabilities P_z . Today's z is observed and the simulator hands back z' ; the matrix is never shown, and the lab raises if one is read.
- ▶ **Shown:** the feasible set and a realised reward — not $u(\cdot)$, not f .
- ▶ **Kept by the grader:** the exact V^*, Q^*, g^* . Three words, fixed for the session — that triple is the **answer key**, the loss against it the **grade**, dynamic programming the **benchmark**.
- ▶ **Hiding a P_z that exists is the weakest form of not knowing**, and the book does it on purpose so a learner can be graded state by state. The real case is the previous frame.

Source: Miao & Feng, *AI for Economists*, ch. 11, §11.2.3.

The Answer to Challenge 2: Use Only What Can Be Observed



all three emit the same four numbers

- ▶ A coin's bias is learned by tossing it; a shock process the same way.
- ▶ Nobody can write Γ_g down, yet a simulated panel can be stepped forward.
- ▶ The requirement is weaker than "know the model": one transition at a time.
- ▶ **Every algorithm here reads only realised (s, a, r, s') — state, action, reward, next state. That is what makes challenge 2 passable.**

Source: Krusell & Smith, *JPE* 106(5), 1998; Maliar, Maliar & Winant, *JME* 122, 2021; Miao & Feng, ch. 11.

Dynamic Programming and Reinforcement Learning: the Careful Statement

a sampled Bellman update (Deck 3.4 names it)

$$Q(s, a) \leftarrow Q(s, a) + \eta [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

↓ expectation taken exactly, step size $\eta = 1$

value function iteration $Q(s, a) \leftarrow R(s, a) + \gamma \sum_{s'} P(s' | s, a) \max_{a'} Q(s', a')$

- ▶ **Same problem:** both find the fixed point of the same Bellman operator.
- ▶ **Same update, in the limit.** Take the expectation exactly and dynamic programming is what is left — the corner with no approximation of any kind.
- ▶ **What differs is what the update may read** — P , or realised transitions, which is why the sampled form survives challenge 2.
- ▶ **What does not carry over: the guarantees.** Contraction and finite termination belong to the *exact* operator. **Which survive sampling is Deck 3.2's question.**

Source: D. Bertsekas, J. Tsitsiklis, *Neuro-Dynamic Programming*, 1996; J. Tsitsiklis, *Machine Learning* 16, 1994; Sutton & Barto, 2018, ch. 4.

Session 3.1

One Problem, Two Languages

And the growth model, in the two instances this session runs



The Markov Decision Process

- ▶ A **Markov decision process** (MDP) is a five-tuple

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle.$$

- ▶ **States** $\mathcal{S}$; feasible **actions** $\mathcal{A}(s)$; **transition** probabilities $P(s' \mid s, a)$; expected **reward** $R(s, a)$; **discount factor** $\gamma \in [0, 1)$.
- ▶ The **Markov property**: the current state carries everything needed to forecast,

$$P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots) = P(s_{t+1} \mid s_t, a_t).$$

- ▶ Every model in this session is an MDP. What changes from deck to deck is which parts of the five-tuple an algorithm is allowed to read.
- ▶ **From here on γ is the discount factor: the economist's β , nothing else.**

Source: M. Puterman, *Markov Decision Processes*, 1994; Miao & Feng, *AI for Economists*, ch. 11.

When the State Is Not Markov, Enlarge It

- ▶ Whether a problem is Markov depends on what you put in the state: it is a property of the process *and* the state description together, never of the description alone.
- ▶ Habit formation: utility depends on c_{t-1} , so (k, z) no longer forecasts the future. Add c_{t-1} to the state and the property returns.
- ▶ The price is dimension — one axis per remembered variable, which is challenge 1 again. If part of the state is never observed, the problem is a **partially observable** MDP: one acts on a belief.
- ▶ When *no* choice of natural variables works — a non-optimal economy may admit no continuous recursive equilibrium — carry the **continuation values** in the state instead.
- ▶ The Markov property makes a problem *recursive*. It does not make it *computable*.
- ▶ **Choosing the state is a modelling decision. Sometimes the repair is to carry a value as a state — which is exactly what a critic does.**

Source: Z. Feng, J. Miao, A. Peralta-Alva & M. Santos, *International Economic Review* 55(1), 2014; Santos, *JET* (2002), shown in Session 1.4; Kaelbling, Littman & Cassandra, *Artificial Intelligence* 101, 1998.

One Dictionary: the MDP and the Dynamic Economic Model

MDP	Economics	Example
state s	state variables	capital k , wealth, productivity z
action a	control, decision	next capital k' , the policy rate
transition $P(s' s, a)$	law of motion, shocks	$k' = a; \ln z' = \rho \ln z + \eta'$
reward $R(s, a)$	period utility or profit	$u(c)$
discount γ	discount factor β	0.99 quarterly, 0.95 annual
policy $g(a s)$	decision rule	$k' = g(k, z)$
value $V^g(s)$	value function	lifetime utility $V(k, z)$

- ▶ The book writes the policy g ; the RL literature writes π . This session follows the book.
- ▶ **Rule, policy, decision rule** are one object; an **actor** is a parameterised one, g_θ .
Session 2's share s is here the action a , since s is a state.

Source: Miao & Feng, *AI for Economists*, ch. 11, "The MDP and dynamic economic models".

The Same Equation in Two Languages

economics $V(k, z) = \max_{k'} \{u(c) + \beta E[V(k', z') | z]\}, \quad c = zk^\alpha + (1 - \delta)k - k'$



reinforcement learning $V^*(s) = \max_{a \in \mathcal{A}(s)} \{R(s, a) + \gamma \sum_{s'} P(s' | s, a) V^*(s')\}$

- ▶ Both say: the value of a state is today's payoff plus the discounted expected value of tomorrow's state.
- ▶ Only the names differ. k is a state, k' is an action, P_z is the transition, β is γ .
- ▶ When the table is too large, both fields reach for function approximation.
- ▶ **The Bellman backbone never changes this session. Only how its expectation is filled.**

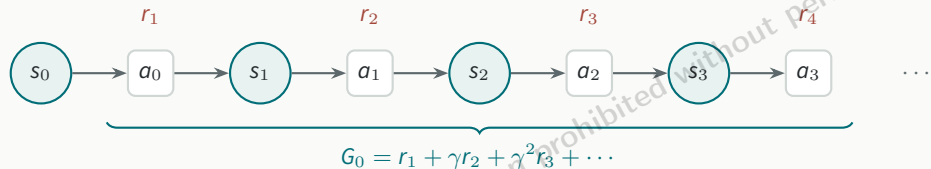
Source: R. Bellman, *Dynamic Programming*, 1957; Miao & Feng, *AI for Economists*, ch. 11.

Policy: a Rule, Deterministic or Random

- ▶ A **policy** maps states to actions. It is the economist's decision rule.
- ▶ **Deterministic**: $a = g(s)$. The optimal saving function $k' = g(k, z)$ is one.
- ▶ **Stochastic**: $a \sim g(\cdot | s)$, a distribution over the feasible actions in each state. **Deck 3.2's rule is the deterministic branch** — a saving rule, as this model writes one. It draws *around* that rule only to find out what else was worth trying.
- ▶ Economists rarely need random rules: with a known model the optimum is deterministic.
- ▶ A learner does need them: samples show only the actions you took, so a rule that never tries the second-best action never learns whether it was second-best.
- ▶ **Randomness in the policy is not noise in behaviour. It is how the data get collected.**

Source: Miao & Feng, *AI for Economists*, ch. 11, “Policy, return and value functions”.

Trajectory and Return



- ▶ A **trajectory** is one simulated path. The reward r_{t+1} is period t 's utility $u(c_t)$; the index says when it is seen.
- ▶ The **return** is the discounted utility stream, $G_t = \sum_{j \geq 0} \gamma^j r_{t+j+1}$.
- ▶ It is recursive: $G_t = r_{t+1} + \gamma G_{t+1}$. Take expectations and the Bellman equation appears.
- ▶ **A return is one random draw of lifetime utility. A value is its expectation.**

Source: Sutton & Barto, *Reinforcement Learning*, 2018, §3.3; Miao & Feng, *AI for Economists*, ch. 11.

Two Value Functions: V and Q

- ▶ The **state-value function**: expected return from state s when following g ,

$$V^g(s) = E_g[G_t \mid s_t = s].$$

- ▶ The **action-value function**, or **Q function**: take action a first, follow g afterwards,

$$Q^g(s, a) = E_g[G_t \mid s_t = s, a_t = a].$$

- ▶ Linked by $V^g(s) = \sum_a g(a \mid s) Q^g(s, a)$. The **Bellman expectation equation** evaluates a *given* rule, with no max. Call its right-hand side $T^g V$:

$$V^g(s) = R(s, g(s)) + \gamma \sum_{s'} P(s' \mid s, g(s)) V^g(s').$$

- ▶ T^g is a contraction of modulus γ , exactly as T is. The max is what made T hard to apply without the model; T^g has no max — and Deck 3.2 shows that this is precisely the guarantee that survives when the model goes.

Source: Miao & Feng, *AI for Economists*, ch. 11, “The recursive form of the principle of optimality”.

Why a Learner Keeps Q — and What Happens When It Cannot

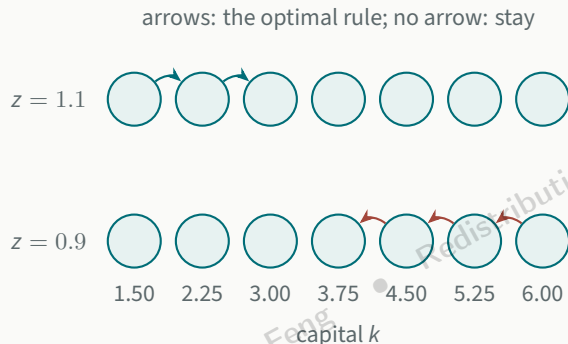
	next capital k'				
	1.50	2.25	3.00	3.75	4.50
$k = 3.75$	1.88	2.90	3.51	3.74	3.05

one row of Q : the largest entry is the improved rule

- ▶ With V^g : comparing actions needs $R + \gamma \sum_{s'} P V^g$. **The model is back.**
- ▶ With Q^g : one row, the expectation already inside the samples — **that is how a learner improves a rule without a model.**
- ▶ **But that is the grid.** On the continuous model k and the share are both real: no cell, no finite row. A max is still available over a fine grid of shares — as the key takes — but only with the model, or a learned $\hat{Q}$ (Deck 3.5).
- ▶ **A gradient needs neither, and it survives Session 4.**

Source: Exact Q^* at $(k, z) = (3.75, 0.9)$, 7×2 grid. Miao & Feng, ch. 11.

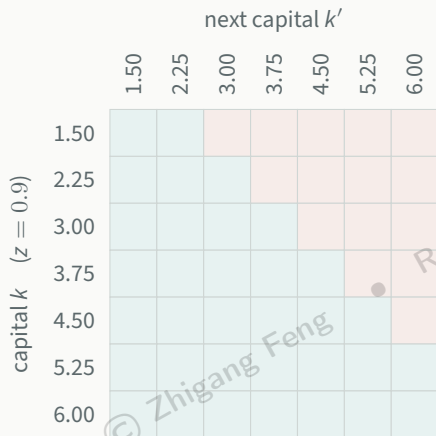
Two Instances of One Model



- ▶ **The 7×2 grid** (drawn): 14 states, the action is next capital k' ; every value can be printed. **Decks 3.3–3.4 run here.**
- ▶ **The continuous model** (frame 3): k real, the action the share $a \in (0, 1)$, the key a 400-point DP. **Deck 3.2 runs here.**
- ▶ **The action is not the same object in the two:** on the grid it is k' , here the share. “Equivalently” holds for the model, not for the action space.

Source: Miao & Feng, ch. 11, “The benchmark model”; rule from the Session 3 lab, block A.

Feasible Actions and the Action Mask



Source: Session 3 lab, block A; 7×2 grid, $\alpha = 0.36$, $\beta = 0.95$, $\delta = 0.10$, $u = \ln c$, $z \in \{0.9, 1.1\}$ with switching probability 0.2.

- ▶ Consumption must be positive, so the feasible set $\mathcal{A}(k, z)$ depends on the state.
- ▶ Teal cells are feasible: 34 at each z , 68 pairs in all.
- ▶ An **action mask** enforces it: infeasible entries get $-\infty$ before any $\arg \max$.
- ▶ **The continuous instance needs no mask at all:** $k' = ay$ with $a \in (0, 1)$ keeps consumption positive by construction. That is Session 2's sigmoid share, and it is why Deck 3.2 never checks feasibility at all.

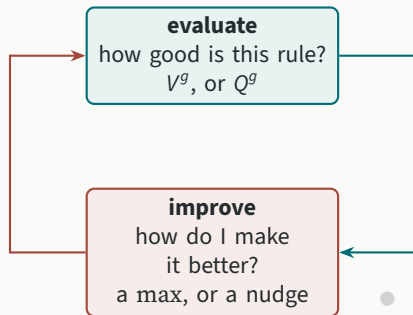
The Environment as Code: `reset` and `step`

```
env = make_env_7x2(seed=0)
state = env.reset(k_idx=3, z_idx=1)      # start at k = 3.75, z = 1.1
mask = env.action_mask(state)           # which k' leave consumption positive
state, reward, terminated, truncated, info = env.step(3)      # choose k' = 3.75
# -> state (3, 1) or (3, 0), reward = ln(1.3953) = 0.3331, terminated = False
```

- ▶ `step` draws tomorrow's productivity inside the simulator. The learner gets the draw, never the probabilities.
- ▶ `terminated` is always false (infinite horizon); `truncated` ends a simulated segment. Treating truncation as the end of time would value all later saving at zero.
- ▶ The continuous instance has *the same two methods*: `reset` puts you somewhere in the band, `step` takes a share and returns r and s' .
- ▶ **This interface is all any algorithm in the lab may touch.**

Source: Book code, ch. 11 (the model) and ch. 12 (the interface), as vendored in the Session 3 lab, block A.

Two Questions Alternate All Session



every method here is a choice of how to answer each, and how far to run one before switching

- ▶ Run *evaluate* to convergence and then improve, and you have **policy iteration**. Take one backup instead and you have value function iteration, which Session 1.4 gave you.
- ▶ Policy iteration appears nowhere in Sessions 1 or 2. Deck 3.2 defines it in one sentence, because its loop is the one every learner in this session runs.
- ▶ **Deck 3.2 answers both at once — on the continuous model, with P_z hidden, and with eight numbers.**

Source: Miao & Feng, *AI for Economists*, ch. 11 — the two alternating questions; R. Howard, *Dynamic Programming and Markov Processes*, 1960.